

TRƯỜNG ĐẠI HỌC CÔNG NGHỆ THÔNG TIN, ĐHQG-HCM

KHOA KỸ THUẬT MÁY TÍNH



BÁO CÁO ĐỒ ÁN MÔN HỌC

ĐỀ TÀI: <XE CÂN BẰNG MINI>

Môn học: <CE224.Q12> - <Thiết kế hệ thống nhúng>

Giảng viên hướng dẫn: <Trần Ngọc Đức>

Thực hiện bởi nhóm <STT Nhóm>, bao gồm:

- | | |
|--------------------|----------|
| 1. Nguyễn Thế Vĩ | 23521775 |
| 2. Lê Tuấn Anh | 23520049 |
| 3. Nguyễn Tiến Văn | 22521651 |
| 4. Trần Thiên Đạt | 23520274 |

Thời gian thực hiện: <1/12/2025> - <23/12/2025>

MỤC LỤC

Chương 1. TỔNG QUAN.....	5
1.1 Giới thiệu đề tài.....	5
1.2. Mục tiêu nghiên cứu.....	5
1.3 Phạm vi nghiên cứu.....	5
Chương 2.CƠ SỞ LÝ THUYẾT.....	6
2.1 Nguyên lý cân bằng động.....	6
2.1.1 Mô hình con lắc ngược.....	6
2.1.2 Đặc điểm động học.....	6
2.1.3 Tính chất phi tuyến.....	6
2.1.4 Nguyên lý giữ thăng bằng.....	6
2.1.5 Thách thức kỹ thuật.....	7
2.2 Hệ thống điều khiển tự động.....	7
2.3 Bộ điều khiển PID.....	7
2.3.1 Nguyên lý bộ điều khiển PID.....	7
2.3.2 Sơ đồ khối và Quy trình điều khiển (Workflow).....	8
2.3.3 Các giai đoạn thực hiện thuật toán.....	8
2.4 Cảm biến IMU (Inertial Measurement Unit).....	10
2.5 Encoder động cơ.....	10
Chương 3. THIẾT KẾ HỆ THỐNG.....	11
3.1. Thiết kế phần cứng và sơ đồ kết nối.....	11
3.2. Thực nghiệm và xác định thông số PID Tốc độ (Speed Loop):.....	11
3.2.1 Phương pháp xác định thông số PI bằng phương pháp Ziegler-Nichols Vòng hở - Open Loop.....	11
3.2.2 Cấu hình file .ioc và lập trình trên STM32.....	13
3.3. Xây dựng chương trình tổng thể và tinh chỉnh PID Góc (Angle Loop):.....	20
3.3.1 Thuật toán xử lý góc nghiêng và cấu trúc điều khiển tầng (Cascade).....	20
3.3.2 Cấu hình file .ioc và tích hợp ngoại vi bổ sung.....	21
3.3.3 Giải thích mã nguồn chương trình chính.....	21
3.3.4 Tinh chỉnh thông số và Kết quả thực nghiệm.....	26
Chương IV. KẾT LUẬN.....	26
NGUỒN THAM KHẢO.....	27

TÓM TẮT

Nội dung tóm lược về đề tài đồ án môn học của nhóm thực hiện sẽ được viết ở đây. Khuyến khích viết ngắn gọn, không đi sâu vào chi tiết, và liệt kê được các điểm quan trọng trong nội dung của đồ án và các kết quả mà nhóm đạt được.

Chương 1. TỔNG QUAN.

1.1 Giới thiệu đề tài.

Xe cân bằng mini là một mô hình robot hai bánh có khả năng **tự giữ thăng bằng** nhờ vào việc sử dụng cảm biến quán tính (IMU) kết hợp với vi điều khiển để xử lý tín hiệu và điều khiển động cơ.

1.2. Mục tiêu nghiên cứu.

- Thiết kế và chế tạo một xe cân bằng mini có khả năng đứng thẳng trên hai bánh.
- Ứng dụng **vi điều khiển STM32F407** để xử lý dữ liệu cảm biến và điều khiển động cơ.
- Xây dựng **thuật toán PID** nhằm giữ xe ổn định và có thể di chuyển tiến/lùi.
- Kiểm chứng hoạt động qua **mô phỏng** và **thực nghiệm thực tế**.

1.3 Phạm vi nghiên cứu

- Xe mô hình nhỏ, sử dụng động cơ DC có encoder.
- Tập trung vào khả năng cân bằng và điều khiển cơ bản trên mặt phẳng.

Chương 2. CƠ SỞ LÝ THUYẾT.

2.1 Nguyên lý cân bằng động

Xe cân bằng hai bánh là một ứng dụng điển hình của mô hình **con lắc ngược** – một hệ thống cơ học vốn không ổn định, luôn có xu hướng ngã về một phía khi không có tác động điều khiển. Để xe có thể đứng thẳng, cần có cơ chế điều khiển liên tục nhằm đưa trọng tâm trở về vị trí cân bằng.

2.1.1 Mô hình con lắc ngược

- Xe cân bằng mini có thể được xem như một **con lắc ngược đặt trên xe hai bánh**.
- Khi xe nghiêng một góc θ so với phương thẳng đứng, trọng lực tạo ra mô-men khiến xe tiếp tục ngã.
- Động cơ phải quay bánh xe theo hướng ngược lại để sinh ra lực phản hồi, đưa xe trở về vị trí cân bằng.

2.1.2 Đặc điểm động học

- Trọng tâm**: nằm phía trên trục bánh xe, càng gần trục thì xe càng dễ giữ thẳng bằng.
- Góc nghiêng θ** : là thông số quan trọng nhất cần đo và điều khiển.
- Tốc độ góc $\dot{\theta}$** : cho biết xu hướng ngã nhanh hay chậm, dùng để dự đoán và điều chỉnh kịp thời.
- Lực điều khiển**: được tạo ra bởi động cơ DC thông qua tín hiệu PWM từ vi điều khiển.

2.1.3 Tính chất phi tuyến

- Hệ thống xe cân bằng là **phi tuyến** vì quan hệ giữa góc nghiêng và lực điều khiển không tuyến tính.
- Để thuận tiện cho thiết kế bộ điều khiển, hệ thống thường được **tuyến tính hóa quanh vị trí cân bằng** (góc $\theta \approx 0$).
- Sau khi tuyến tính hóa, có thể áp dụng các phương pháp điều khiển kinh điển như PID hoặc LQR.

2.1.4 Nguyên lý giữ thẳng bằng

- Khi xe nghiêng về phía trước: động cơ quay bánh xe tiến lên để đưa trọng tâm trở lại.
- Khi xe nghiêng về phía sau: động cơ quay bánh xe lùi lại để cân bằng.
- Quá trình này diễn ra liên tục, với tốc độ phản ứng nhanh, nhờ vào việc đọc dữ liệu cảm biến và điều chỉnh động cơ theo thuật toán điều khiển.

2.1.5 Thách thức kỹ thuật

- Độ trễ cảm biến:** dữ liệu từ IMU có nhiễu, cần lọc tín hiệu.
- Độ chính xác điều khiển:** hệ số PID phải được tinh chỉnh phù hợp để xe không rung lắc hoặc ngã.
- Tác động môi trường:** mặt sàn không bằng phẳng, ma sát bánh xe, và rung động đều ảnh hưởng đến khả năng cân bằng.

2.2 Hệ thống điều khiển tự động

-Khái niệm: Hệ thống điều khiển tự động là hệ thống có khả năng tự điều chỉnh đầu ra dựa trên tín hiệu đầu vào và trạng thái hiện tại.

-Thành phần cơ bản:

- +Cảm biến đo trạng thái (IMU, encoder).
- +Bộ điều khiển (vi điều khiển + thuật toán).
- +Cơ cấu chấp hành (động cơ, driver).
- +Đối tượng điều khiển (xe cân bằng).

-Nguyên lý vòng kín: tín hiệu phản hồi từ cảm biến được đưa về bộ điều khiển để tính toán và điều chỉnh đầu ra, giúp hệ thống ổn định.

2.3 Bộ điều khiển PID

2.3.1 Nguyên lý bộ điều khiển PID

Bộ điều khiển PID (Proportional-Integral-Derivative) là một cơ chế phản hồi vòng kín được sử dụng rộng rãi trong công nghiệp và robot. Đối với hệ thống xe cân bằng, PID đóng vai trò xử lý sai số giữa trạng thái thực tế và trạng thái mong muốn để đưa ra tín hiệu điều khiển động cơ phù hợp.

- **P (Proportional - Hệ số tỷ lệ):** Phản ứng trực tiếp với sai số hiện tại. Sai số càng lớn, lực đẩy điều chỉnh càng mạnh.
- **I (Integral - Hệ số tích phân):** Tích lũy sai số theo thời gian, giúp loại bỏ sai số tĩnh (offset), đảm bảo xe đứng thẳng tuyệt đối ở trạng thái cân bằng.

- **D (Derivative - Hệ số vi phân):** Phản ứng theo tốc độ thay đổi của sai số, đóng vai trò như một bộ giảm chấn giúp hệ thống ổn định, tránh dao động quá mức.

- **Ưu điểm:** Dễ cài đặt, cấu trúc đơn giản nhưng mang lại hiệu quả cao với các hệ thống được tuyến tính hóa như xe cân bằng.

- **Hạn chế:** cần tinh chỉnh hệ số Kp, Ki, Kd để đạt hiệu quả tối ưu.

2.3.2 Sơ đồ khối và Quy trình điều khiển (Workflow)

Hệ thống điều khiển xe cân bằng được thiết kế theo mô hình **vòng lặp kép (Cascade Control)** để đảm bảo xe vừa giữ được thăng bằng, vừa kiểm soát được vị trí/tốc độ:

Góc đo (α) $\xrightarrow{\text{PID Ngoài}}$ Tốc độ cần đạt $\xrightarrow{\text{PID Trong}}$ Xung PWM $\rightarrow$ Động cơ

2.3.3 Các giai đoạn thực hiện thuật toán

Giai đoạn 1: Tính toán góc nghiêng thực tế (Alpha_meas)

Mục tiêu: Biến dữ liệu nhiễu của cảm biến thành một con số góc mượt mà.

1. Tính góc từ Gia tốc kế (Accelerometer):

$$\text{Alpha_acc} = \text{atan2}(\text{Ay}, \text{Az}) * (180 / 3.14)$$

(Trong đó Ay, Az là giá trị đọc thô từ MPU6050).

2. Bộ lọc bù (Complementary Filter) để hợp nhất với Gyro:

$$\text{Alpha_hien_tai} = 0.98 * (\text{Alpha_truoc} + \text{Wx} * \text{Delta_t}) + 0.02 * \text{Alpha_acc}$$

- Wx: Tốc độ góc từ Gyroscope (độ/giây).
- Delta_t: Thời gian lấy mẫu (ví dụ 0.01s).
- 0.98/0.02: Hệ số tin tưởng (ưu tiên Gyro để tránh nhiễu tức thời).

Giai đoạn 2: Vòng PID Ngoài (Angle Loop)

Mục tiêu: Biến lỗi góc nghiêng thành Tốc độ mong muốn (Target Speed).

1. Tính lỗi góc:

$$\text{E_angle} = \text{Setpoint_angle} - \text{Alpha_hien_tai}$$

(Thường Setpoint_angle xấp xỉ 0 để xe đứng thẳng).

2. Tính toán PID:

- $P_out = Kp1 * E_angle$
- $I_out = I_out_truoc + (Ki1 * E_angle * Delta_t)$
- $D_out = Kd1 * (E_angle - E_angle_truoc) / Delta_t$

1. Kết quả đầu ra:

$$Target_Speed = P_out + I_out + D_out$$

Giai đoạn 3: Vòng PID Trong (Speed Loop)

Mục tiêu: Biến lỗi tốc độ thành giá trị PWM.

1. Đọc tốc độ thực tế từ Encoder:

$$V_thuc = (Encoder_moi - Encoder_cu) / Delta_t$$

2. Tính lỗi tốc độ:

$$E_speed = Target_Speed - V_thuc$$

3. Tính toán PID (Thường chỉ cần PI):

- $P_spd = Kp2 * E_speed$
- $I_spd = I_spd_truoc + (Ki2 * E_speed * Delta_t)$

1. Kết quả PWM thô:

$$PWM_out = P_spd + I_spd$$

Giai đoạn 4: Xử lý phần cứng (Hardware Mapping)

Mục tiêu: Đưa PWM_out vào Driver động cơ (TB6612).

1. Giới hạn (Constrain): Đảm bảo PWM không vượt quá giới hạn Timer (ví dụ tối đa 1000).

$$PWM_final = constrain(PWM_out, -1000, 1000)$$

2. Điều khiển chiều (Logic Driver):

- Nếu $PWM_final > 0$:
- $DIR_PIN_1 = HIGH, DIR_PIN_2 = LOW$ (Quay tới)
- $PWM_PIN =$ Giá trị tuyệt đối của PWM_final
- Nếu $PWM_final < 0$:
- $DIR_PIN_1 = LOW, DIR_PIN_2 = HIGH$ (Quay lùi)
- $PWM_PIN =$ Giá trị tuyệt đối của PWM_final

2.4 Cảm biến IMU (Inertial Measurement Unit)

-Thành phần chính:

+**Gia tốc kế (Accelerometer)**: đo gia tốc tuyến tính theo các trục X, Y, Z. Từ đó có thể suy ra góc nghiêng của xe so với phương thẳng đứng.

+**Con quay hồi chuyển (Gyroscope)**: đo tốc độ góc quay quanh các trục. Dữ liệu này cho biết xe đang nghiêng nhanh hay chậm.

-**Ví dụ cảm biến**: MPU-6050, tích hợp cả gia tốc kế và gyro, giao tiếp qua chuẩn I²C.

-**Vai trò trong hệ thống**: cung cấp thông tin về trạng thái cân bằng (góc nghiêng, tốc độ góc) để bộ điều khiển tính toán và điều chỉnh động cơ.

2.5 Encoder động cơ

-**Nguyên lý**: Encoder gắn trên động cơ cung cấp tín hiệu xung theo vòng quay.

-Ứng dụng:

+Vi điều khiển đọc xung encoder để tính tốc độ và quãng đường bánh xe.

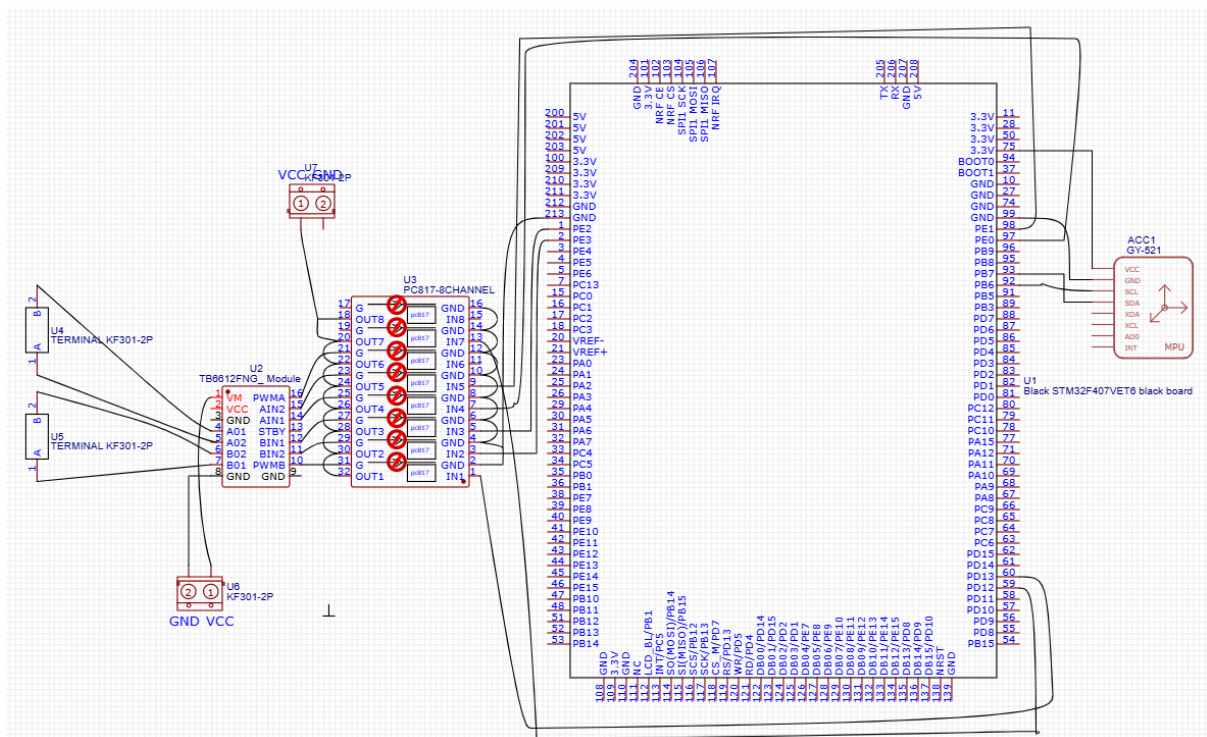
+Dữ liệu này dùng cho PID tốc độ và để xe di chuyển theo lệnh.

-**Ưu điểm**: đo chính xác tốc độ và vị trí, hỗ trợ điều khiển chính xác.

Chương 3. THIẾT KẾ HỆ THỐNG.

3.1. Thiết kế phần cứng và sơ đồ kết nối:

- Thiết bị và linh kiện:
 - + STM32F407Vet6 board
 - + TB6612FNG
 - + 2 động cơ DC 6-12 V
 - + Module PC817 để chống nhiễu và bảo vệ MCU.
 - + Cáp USB type A- micro,dây cảm, breadboard
 - + MPU6050
- Sơ đồ kết nối:



3.2. Thực nghiệm và xác định thông số PID Tốc độ (Speed Loop):

Trong hệ thống này, việc xác định các thông số PI không chỉ dựa vào thử sai thuần túy mà được thực hiện thông qua phương pháp thực nghiệm khoa học để tìm ra mô hình toán học của động cơ.

3.2.1 Phương pháp xác định thông số PI bằng phương pháp Ziegler-Nichols Vòng hở - Open Loop

-Để tính toán bộ điều khiển PI, nhóm sử dụng một phần mềm bên ngoài để vẽ đồ thị so sánh giữa tốc độ mong muốn (Desired Speed) và tốc độ thực tế (Current Speed).

-Quy trình xác định các thông số K_p và K_i được thực hiện qua các bước sau:

Bước 1: Tính Hệ số khuếch đại của quá trình (K)

Hệ số K cho biết mức độ thay đổi vận tốc thực tế khi thay đổi đầu ra motor (PWM).

Công thức: $K = \Delta_{CV} / \Delta_{MV}$

Trong đó:

- Δ_{MV} : Độ thay đổi của biến điều khiển (Ví dụ: tăng PWM thêm 100 đơn vị).
 - Δ_{CV} : Độ thay đổi của vận tốc thực tế đo được từ Encoder (Ví dụ: vận tốc tăng thêm 20 rad/s).
- $\Rightarrow K = 20 / 100 = 0.2.$

Bước 2: Xác định các thông số thời gian từ đồ thị

Quan sát đồ thị vận tốc theo thời gian để xác định hai thông số đặc trưng của hệ thống:

1. Thời gian trễ (L hoặc Theta): Khoảng thời gian từ lúc lệnh PWM thay đổi đến lúc vận tốc bắt đầu có sự chuyển biến thực tế.
2. Hằng số thời gian (Tau): Khoảng thời gian tính từ lúc vận tốc bắt đầu thay đổi cho đến khi đạt được 63.2% giá trị xác lập mới.

Bước 3: Tính toán thông số PI theo quy tắc Ziegler-Nichols (Vòng hở)

Dựa trên mô hình nấc (Step Response) đã xác định ở Bước 1 và 2, các thông số của bộ điều khiển được tính toán theo công thức:

- Hệ số khuếch đại (K_p): $K_p = 0.9 * (1 / K) * (Tau / L)$
- Thời gian tích phân (T_i): $T_i = 3.3 * L$

Bước 4: Chuyển đổi sang mã code trên vi điều khiển STM32

Trong chương trình điều khiển, bộ tham số PI được áp dụng trực tiếp như sau:

- K_p = giá trị K_p tính được ở trên.
- $K_i = K_p / T_i$.

Thuật toán triển khai rời rạc:

Tại mỗi chu kỳ lấy mẫu T (ví dụ 10ms), tín hiệu điều khiển được tính toán:

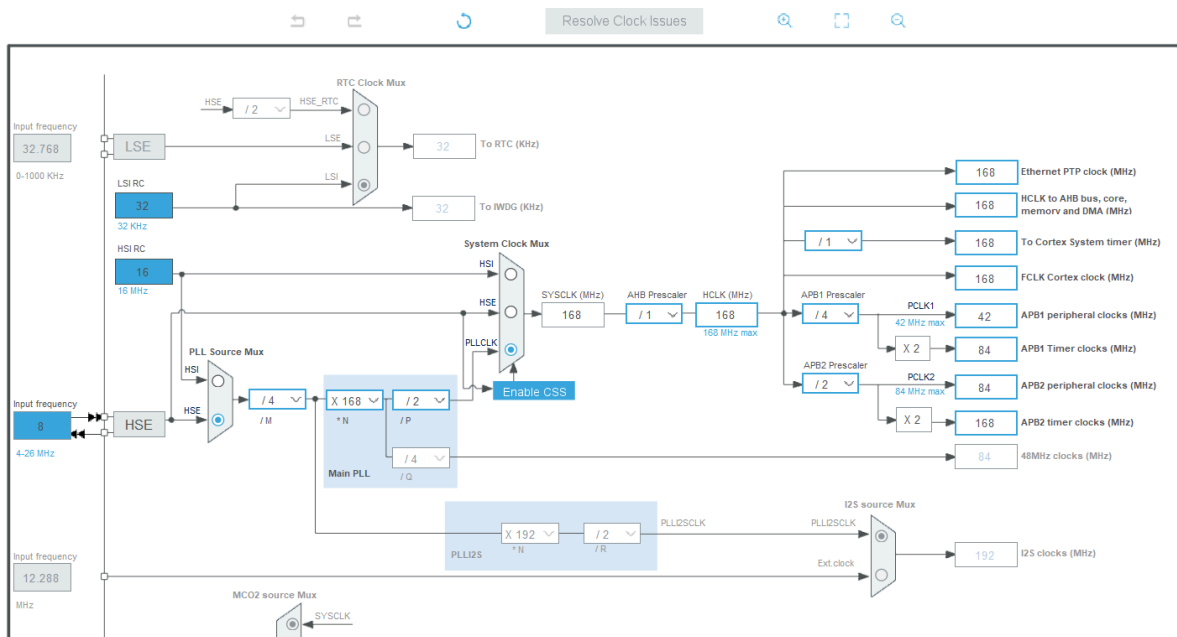
1. Sai số: $err = \text{Tốc độ đặt} - \text{Tốc độ thực tế}$.
2. Thành phần tỷ lệ: $up = K_p * err$.
3. Thành phần tích phân: $ui = ui_trước + K_i * err * 0.01$.
4. Tổng đầu ra: $uout = up + ui$.

Giá trị $uout$ cuối cùng sẽ được đưa qua bộ giới hạn (Saturation) để đảm bảo không vượt quá ngưỡng PWM của Timer, sau đó xuất tín hiệu qua module cách ly PC817 để điều khiển Driver TB6612.

3.2.2 Cấu hình file .ioc và lập trình trên STM32

1. Cấu hình trên phần mềm STM32CubeIDE (.ioc)

- + **Hệ thống Clock (RCC):** Cấu hình thạch anh ngoài 8MHz, nhân tần qua bộ PLL để đạt tốc độ xử lý tối đa của chip là 168MHz. Điều này giúp các phép toán số thực (float) trong PID được thực thi cực nhanh.



+ Bộ đếm Encoder (TIM2 & TIM3):

Mode: Encoder Mode (TI1 and TI2).

Thông số: Counter Period được đặt tối đa (4,294,967,295 cho TIM2 - 32 bit).

Bộ lọc (Filter): Đặt giá trị 10 để loại bỏ nhiễu điện từ phát ra từ motor 12V, giúp kết quả đếm xung chính xác hơn.

+ Phát xung PWM (TIM4):

Prescaler: 83, *Period (ARR):* 999. Với xung nhịp 84MHz từ APB1, tần số PWM đầu ra đạt mức 1kHz, phù hợp để điều khiển motor mà Driver TB6612.

Channel: Kích hoạt Channel 1 và Channel 2 để điều khiển hai động cơ.

+ Ngắt chu kỳ mẫu (TIM5):

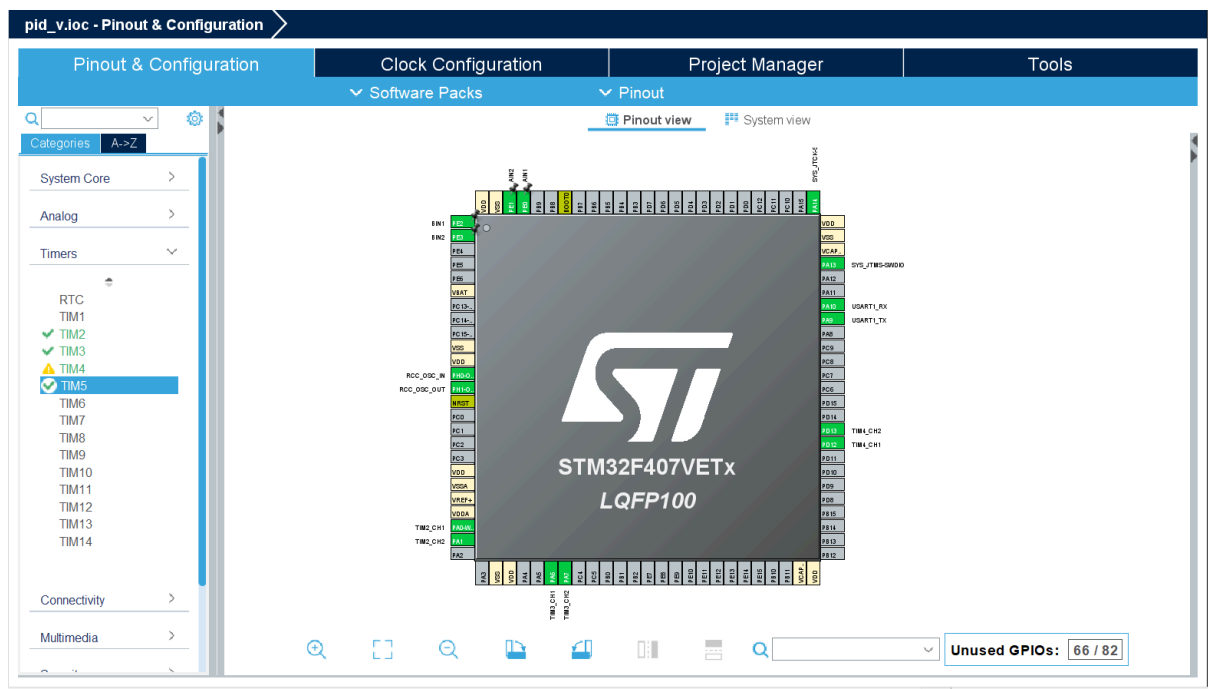
Prescaler: 8399, *Period:* 99, và kích hoạt "TIM5 global interrupt" để cho phép vi điều khiển nhảy vào trình phục vụ ngắt định kỳ. Cấu hình này tạo ra một ngắt định kỳ đúng **10ms (100Hz)**. Đây là khoảng thời gian lấy mẫu

(SAMPLE_TIME) cố định cho thuật toán PI, đảm bảo tính ổn định của hệ thống điều khiển.

+ **Giao tiếp UART (USART1):**

Baudrate: 115200. Kích hoạt ngắt nhận dữ liệu (UART RX Interrupt) để nhận lệnh điều khiển vận tốc từ máy tính hoặc điện thoại.

+ **Cấu hình GPIO:** Các chân GPIOE (BIN1, BIN2, AIN1, AIN2) được thiết lập ở chế độ Output Speed High để điều khiển hướng động cơ nhanh chóng thông qua module cách ly PC817.



2. Các hàm và thuật toán lập trình thêm vào

- **Hàm tính toán PID (PIDVel):** Thực hiện tính toán sai số (error = DesiredSpeed - CurrentSpeed). Thành phần tích phân (ui) được tích lũy theo thời gian mẫu 0.01s. Đặc biệt, hàm có tích hợp kỹ thuật Anti-Windup: Nếu đầu ra uout vượt quá giới hạn PWM (999), thành phần tích phân sẽ ngừng cộng dồn để tránh hiện tượng vọt lố quá mức khi hệ thống bị kích trần công suất.

```
int PIDVel(float DesiredValue, float CurrentValue) {
```

```
    err = DesiredValue - CurrentValue;
```

```
    // 1. Thành phần P
```

```
up = Kp * err;
// 2. Thành phần D
ud = Kd * (err - err_p) / SAMPLE_TIME;
// 3. Thành phần I + Chống Windup
ui = ui_p + Ki * err * SAMPLE_TIME;
uout = (int)(up + ud + ui);
// 4. Giới hạn đầu ra và Anti-Windup
if (uout > HILIM) {
    uout = HILIM;
    ui = ui_p; // Không cộng dồn I nữa nếu đã kích trần
} else if (uout < LOLIM) {
    uout = LOLIM;
    ui = ui_p; // Không cộng dồn I nữa nếu đã kích sàn
}
err_p = err;
ui_p = ui; // Cập nhật lại ui_p sau khi đã kiểm tra giới hạn
return uout;
}
```

- **Hàm ngắt Timer 5 (HAL_TIM_PeriodElapsedCallback):**

Đây là "trái tim" của hệ thống, thực hiện các bước sau mỗi 10ms:

- B1: Đọc số xung Encoder hiện tại bằng hàm __HAL_TIM_GET_COUNTER.
- B2: Reset bộ đếm về 0 để tính toán độ chênh lệch xung trong chu kỳ tiếp theo.
- B3: Quy đổi số xung sang vận tốc góc thực tế (rad/s) theo công thức dựa trên PPR (11 xung) và tỷ số truyền (1/30).
- B4: Gọi hàm PIDVel để lấy giá trị PWM điều khiển.
- B5: Cập nhật hướng quay và xuất xung PWM ra chân điều khiển động cơ.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM5) {
        // 1. Đọc số xung từ Hardware Encoder (TIM2)
        raw_pulses_1 = (int16_t) __HAL_TIM_GET_COUNTER(&htim2);
        __HAL_TIM_SET_COUNTER(&htim2, 0); // Reset để lần sau đọc độ chênh lệch
        total_pulses_1 += raw_pulses_1;
        // 2. Tính vận tốc hiện tại (rad/s)
        // Công thức: (số xung * 2 * PI) / (xung_vòng * tỷ_số_truyền * 4_chế_độ_xung * thời_gian_lấy_mẫu)
    }
}
```

```
CurVel = fabsf( (float)(raw_pulses_1 * 2.0f * PI) / (PPR * GEAR_RATIO * 4.0f *
SAMPLE_TIME) );
// Tính vị trí (rad)
CurPos = (float)(total_pulses_1 * 2.0f * PI) / (PPR * GEAR_RATIO * 4.0f);
// 3. Tính toán PID
//int pwm = 0;
if (run) {
    float a= fabs (DesiredSpeed);
    pwm = PIDVel((float)a, CurVel);
    //pwm = DesiredSpeed;
} else {
    pwm = 0;
}
// 4. Xuất PWM và hướng (Sử dụng cấu hình chân Code 2)
if (DesiredSpeed>0){
    HAL_GPIO_WritePin(GPIOE,GPIO_PIN_0, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE,GPIO_PIN_1, GPIO_PIN_RESET);
    __HAL_TIM_SetCompare(&htim4,TIM_CHANNEL_1,pwm); // set pwm
}
else {
    HAL_GPIO_WritePin(GPIOE,GPIO_PIN_1, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE,GPIO_PIN_0, GPIO_PIN_RESET);
    __HAL_TIM_SetCompare(&htim4, TIM_CHANNEL_1,pwm); // set pwm
}
// 5. Gửi dữ liệu lên máy tính (Mỗi 50ms)
tick++;
if (tick >= 5) {
    tick = 0;
    if(run) {
        printf("V%f\r\n",CurVel);
        printf("P%f\r\n",CurPos);
    }
}
}
```

- **Hàm ngắt nhận UART (HAL_UART_RxCpltCallback):**

Xử lý các ký tự nhận được để thay đổi trạng thái hoạt động (chạy/dừng) và cập

nhật vận tốc mong muốn (DesiredSpeed) từ người dùng theo thời gian thực mà không làm gián đoạn chu kỳ tính toán PID.

```
void HAL_UART_RxCpltCallback(UART_HandleTypeDef *huart) {
    if(huart->Instance == USART1) {
        if(Rx_indx == 0) { memset(Rx_Buffer, 0, sizeof(Rx_Buffer)); }
        switch(Rx_data[0]) {
            case 'e': run = false; break;
            case 'r': run = true; break;
            case 'v': // Set speed
                DesiredSpeed = atoi(Rx_Buffer);
                Rx_indx = 0;
                break;
            case '0': case '1': case '2': case '3': case '4':
            case '5': case '6': case '7': case '8': case '9':
            case '!': case '-':
                if(Rx_indx < 19) Rx_Buffer[Rx_indx++] = Rx_data[0];
                break;
            default: break;
        }
        HAL_UART_Receive_IT(&huart1, (uint8_t*)Rx_data, 1);
    }
}
```

c) Kết quả sau khi tính toán và tinh chỉnh

1. Kết quả đo đặc thực nghiệm:

Nhóm tiến hành thay đổi giá trị đầu ra PWM (khi này PWM = Desired Speed) và đo vận tốc phản hồi (Current Velocity - CurVel) từ Encoder, kết quả thu được như sau:

- Tại mức PWM 1 (DesiredSpeed_1): 50 đơn vị => Vận tốc thực tế (CurVel_1): 1.427496 rad/s.
- Tại mức PWM 2 (DesiredSpeed_2): 100 đơn vị => Vận tốc thực tế (CurVel_2): 5.23598 rad/s.
- Thời gian trễ đo được (L): 0.0769 giây.
- Hằng số thời gian của hệ thống (Tau): 0.08 giây.

2. Tính toán các thông số mô hình:

Dựa trên dữ liệu trên, nhóm tính toán các hệ số cần thiết:

- Độ thay đổi biến điều khiển: $\Delta_{MV} = 100 - 50 = 50$ đơn vị PWM.
- Độ thay đổi biến đầu ra: $\Delta_{CV} = 5.23598 - 1.427496 = 3.808484$ rad/s.
- Hệ số khuếch đại quá trình (K):
$$K = \Delta_{CV} / \Delta_{MV} = 3.808484 / 50 = 0.07617.$$

3. Xác định thông số Kp và Ki theo Ziegler-Nichols:

Áp dụng quy tắc Ziegler-Nichols cho bộ điều khiển PI (Vòng hở):

- Hệ số tỷ lệ (Kp):
$$K_p = 0.9 * (1 / K) * (\tau / L)$$
$$K_p = 0.9 * (1 / 0.07617) * (0.08 / 0.0769) = 12.29.$$
- Thời gian tích phân (Ti):
$$T_i = 3.3 * L = 3.3 * 0.0769 = 0.25377.$$
- Hệ số tích phân (Ki) dùng trong code:
$$K_i = K_p / T_i = 12.29 / 0.25377 = 48.43.$$

4. Đánh giá và tinh chỉnh thực tế:

Sau khi nạp các thông số tính toán lý thuyết ($K_p = 12.29$, $K_i = 48.43$) vào vi điều khiển, nhóm đã tiến hành chạy thực nghiệm và quan sát đồ thị đáp ứng vận tốc.

- **Quan sát ban đầu:** Hệ thống cho kết quả vận tốc thực tế có bám theo được vận tốc đặt (Setpoint). Tuy nhiên, một vấn đề lớn phát sinh là thời gian đáp ứng (Settling time) quá chậm, mất từ 5 đến 10 giây động cơ mới đạt được tốc độ mục tiêu.
- **Phân tích nguyên nhân:** Thời gian phản ứng chậm như trên thường do hệ số khuếch đại còn thấp. Nhóm xác định nguyên nhân có thể đến từ những sai số khách quan trong quá trình quan sát đồ thị thô để đo đạc các thông số thời gian trễ (L và Tau)

- **Quá trình tinh chỉnh (Fine-tuning):** Để khắc phục tình trạng chậm trễ này, nhóm đã quyết định thực hiện phương pháp tinh chỉnh thủ công dựa trên nền tảng số liệu cũ:
- **Kết quả cuối cùng:** Sau nhiều lần thử nghiệm, nhóm đã tìm được bộ thông số cho vòng lặp tốc độ là: $K_p = 18.5$, $K_i = 150$.

Kết luận: Với bộ thông số mới, thời gian đáp ứng của động cơ đã cải thiện rõ rệt, rút ngắn từ 10 giây xuống dưới 1 giây. Động cơ phản ứng gần như tức thời với lệnh điều khiển, vận tốc ổn định và không có hiện tượng vọt lố đáng kể. Kết quả này đảm bảo vòng PID trong (Speed Loop) hoạt động đủ nhanh để có thể hỗ trợ hiệu quả cho vòng PID ngoài (Angle Loop) trong việc giữ thăng bằng cho robot.

Video về kết quả PID tốc độ:

<https://drive.google.com/file/d/1mzwDe13WCkkig9X218WdlwHlrwH9NHJP/view?usp=sharing>

Lưu ý về tính đồng bộ của hệ thống động cơ: Trong quá trình thực nghiệm tại mục 3.2, nhóm đã tiến hành đo đạc và vẽ đồ thị đáp ứng vận tốc cho cả hai động cơ (Trái và Phải). Kết quả cho thấy đồ thị đặc tính động lực học của hai động cơ có sự tương đồng rất cao (sai lệch không đáng kể). Do đó, để tối ưu hóa chương trình điều khiển và đơn giản hóa quá trình tinh chỉnh, nhóm quyết định **sử dụng chung một bộ tham số PID (K_p , K_i)** cho cả hai động cơ. Việc này vẫn đảm bảo robot di chuyển thẳng và phản hồi đồng bộ mà không cần tốn tài nguyên xử lý cho hai bộ PID riêng biệt.

3.3. Xây dựng chương trình tổng thể và tinh chỉnh PID Góc (Angle Loop):

Sau khi đã làm chủ được vòng lặp điều khiển tốc độ (Speed Loop) ở mục 3.2, giai đoạn này nhóm tiến hành tích hợp cảm biến quán tính MPU6050 để xây dựng hệ thống điều khiển thăng bằng hoàn chỉnh cho Robot.

3.3.1 Thuật toán xử lý góc nghiêng và cấu trúc điều khiển tầng (Cascade)

1. Xử lý góc nghiêng bằng Bộ lọc bù (Complementary Filter):

Dữ liệu từ MPU6050 bao gồm Gia tốc kế (Accelerometer) và Con quay hồi chuyển (Gyroscope). Nhóm sử dụng bộ lọc bù để tối ưu hóa dữ liệu:

$$\text{Góc_hiện_tại} = 0.98 * (\text{Góc_cũ} + \text{Tốc_độ_góc} * dt) + 0.02 * \text{Góc_gia_tốc}.$$

Hệ số 0.98 giúp loại bỏ nhiễu rung từ gia tốc kế và 0.02 giúp triệt tiêu hiện tượng trôi (drift) của con quay hồi chuyển theo thời gian.

2. Cấu trúc PID tầng (Cascade PID Control):

Robot sử dụng hai vòng lặp lồng nhau:

- **Vòng Ngoài (Angle Loop):** Đầu vào là sai số góc nghiêng. Đầu ra của vòng này chính là tốc độ mong muốn (Desired Speed) cấp cho vòng trong.
- **Vòng Trong (Speed Loop):** Chính là bộ PI đã thực nghiệm ở mục 3.2, có nhiệm vụ điều khiển động cơ đạt đúng tốc độ mà vòng góc yêu cầu.

3.3.2 Cấu hình file .ioc và tích hợp ngoại vi bổ sung

Để thực hiện chương trình tổng thể, ngoài các cấu hình Encoder và PWM ở mục 3.2, nhóm bổ sung các thiết lập sau trên STM32CubeIDE:

- **I2C1 (Kết nối MPU6050):**

- *Clock Speed:* 400.000 Hz (Fast Mode) để đảm bảo tốc độ đọc dữ liệu cảm biến nhanh nhất.
- *Chân kết nối:* PB6 (SCL), PB7 (SDA).

- **TIM5 (Ngắt chu kỳ 10ms):**

- *Prescaler:* 8399, *Period:* 99.
- *Mục tiêu:* Kích hoạt cờ ready_to_process mỗi 10ms để bắt đầu một chu kỳ tính toán PID đồng bộ.

- **TIM2 (Encoder):** Cấu hình chế độ Encoder Mode để phản hồi tốc độ thực tế từ bánh xe.

- **TIM4 (PWM):** Điều khiển xung PWM cho Driver TB6612 với chu kỳ (Period) 999.

3.3.3 Giải thích mã nguồn chương trình chính.

Mã nguồn được thiết kế theo mô hình máy trạng thái và ngắt, tập trung vào các giai đoạn:

1. Khởi tạo và Hiệu chuẩn (Calibration):

Khi bắt đầu, robot thực hiện nháy LED (chân PA6) để thông báo trạng thái. Hệ thống thực hiện đọc 1000 mẫu dữ liệu từ Gyroscope để tính toán GyroOffset. Việc này giúp triệt tiêu sai số ban đầu của cảm biến khi robot đứng yên.

2. Vòng lặp điều khiển thực thi (While loop):

Khi cờ ready_to_process được bật bởi ngắt TIM5:

- Đọc dữ liệu thô 14 byte từ MPU6050 qua I2C.
- Tính toán góc nghiêng qua bộ lọc bù.
- **PID Góc:** Tính toán các thành phần P, I, D. Trong đó, thành phần D được tính dựa trên GyroRate (tốc độ góc) để tăng độ phản ứng.
- **PID Tốc độ:** Kết quả từ PID Góc được lấy giá trị tuyệt đối (fabs) làm tốc độ mong muốn cho hàm PIDVel.
- **Điều khiển hướng:** Dựa vào dấu của PID_Output để kích hoạt các chân GPIO (AIN1, AIN2, BIN1, BIN2) nhằm quyết định robot tiến hay lùi để giữ thăng bằng.

```
#include "main.h"
#include <math.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
#include <stdbool.h>
/* --- CẤU HÌNH HẲNG SỐ --- */
#define MPU_ADDR (0x68 << 1)
#define RAD_TO_DEG 57.296f
#define PI 3.14159f
#define DT 0.01f
#define SAMPLE_TIME 0.01f
#define PPR 11
#define GEAR_RATIO 30
```

```
/* --- BIẾN TOÀN CỤC --- */
I2C_HandleTypeDef hi2c1;
TIM_HandleTypeDef htim2;
TIM_HandleTypeDef htim4;
TIM_HandleTypeDef htim5;
float TARGET_ANGLE = -1.0f;
float Kp = 0.23f;
float Kd = 0.0f;
float Ki = 0.0f;
float P,D,I;
float Kp_vel = 18.5f;
float Ki_vel = 150.0f;
float Kd_vel = 0.0f;
float up, ui, ud;
int uout;
float CurrentAngle = 0, GyroOffset = 0;
float Error = 0, SumError = 0;
float PID_Output = 0;
float AccAngle, GyroRate;
volatile int16_t raw_pulses_l = 0;
float CurVel = 0, CurPos = 0;
int32_t total_pulses_l = 0;
int16_t HILIM = 999, LOLIM = -999;
int16_t pwm;
volatile bool ready_to_process = false;
/* --- KHAI BÁO PROTOTYPES (BẮT BUỘC CÓ DÒNG NÀY ĐỂ HẾT LỖI) --- */
void SystemClock_Config(void);
static void MX_GPIO_Init(void);
static void MX_I2C1_Init(void);
static void MX_TIM2_Init(void);
static void MX_TIM4_Init(void);
static void MX_TIM5_Init(void);
void MPU6050_Init(void);
int PIDVel(float DesiredValue, float CurrentValue);
/* --- HÀM TÍNH PID TỐC ĐỘ --- */
int PIDVel(float DesiredValue, float CurrentValue) {
    static float err_p = 0;
    static float ui_p = 0;
```

```
float err = DesiredValue - CurrentValue;
up = Kp_vel * err;
ud = Kd_vel * (err - err_p) / SAMPLE_TIME;
ui = ui_p + Ki_vel * err * SAMPLE_TIME;
uout = (int)(up + ud + ui);
if (uout > HILIM) { uout = HILIM; ui = ui_p; }
else if (uout < LOLIM) { uout = LOLIM; ui = ui_p; }
err_p = err;
ui_p = ui;
return uout;
}
/* --- NGẮT TIMER --- */
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim) {
    if (htim->Instance == TIM5) {
        ready_to_process = true;
    }
}
/* --- CHƯƠNG TRÌNH CHÍNH --- */
int main(void) {
    HAL_Init();
    SystemClock_Config();
    MX_GPIO_Init();
    MX_I2C1_Init();
    MX_TIM2_Init();
    MX_TIM4_Init();
    MX_TIM5_Init();
    HAL_TIM_Encoder_Start(&htim2, TIM_CHANNEL_ALL);
    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_1);
    HAL_TIM_PWM_Start(&htim4, TIM_CHANNEL_2);
    HAL_Delay(1000);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);
    HAL_Delay(1000);
    for (int i = 0; i < 20; i++) {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);
        HAL_Delay(100);
    }
    MPU6050_Init();
    HAL_Delay(2000); // Đổi cam bien on dinh
```

```

        for (int i = 0; i < 10; i++) {
            HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_6);
            HAL_Delay(100);
        }
// Hiệu chuẩn Gyro
long gyro_sum = 0;
for (int i = 0; i < 1000; i++) {
    uint8_t data[2];
    if(HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, 0x45, 1, data, 2, 10) == HAL_OK)
        gyro_sum += (int16_t)(data[0] << 8 | data[1]);
    HAL_Delay(1);
}
GyroOffset = (float)gyro_sum / 1000.0f;
HAL_TIM_Base_Start_IT(&htim5);
while (1) {
    if (ready_to_process) {
        ready_to_process = false;
        uint8_t Rec_Data[14];
        if (HAL_I2C_Mem_Read(&hi2c1, MPU_ADDR, 0x3B, 1, Rec_Data, 14, 10) == HAL_OK) {
            int16_t AccX_raw = (int16_t)(Rec_Data[0] << 8 | Rec_Data[1]);
            int16_t AccZ_raw = (int16_t)(Rec_Data[4] << 8 | Rec_Data[5]);
            int16_t GyroY_raw = (int16_t)(Rec_Data[10] << 8 | Rec_Data[11]);
            AccAngle = atan2f((float)AccX_raw, (float)AccZ_raw) * RAD_TO_DEG;
            GyroRate = ((float)GyroY_raw - GyroOffset) / 131.0f;
            CurrentAngle = 0.98f * (CurrentAngle + GyroRate * DT) + 0.02f * AccAngle;

//=====
=====

            raw_pulses_l = (int16_t) __HAL_TIM_GET_COUNTER(&htim2);
            __HAL_TIM_SET_COUNTER(&htim2, 0);
            total_pulses_l += raw_pulses_l;
            CurVel = fabsf((float)(raw_pulses_l * 2.0f * PI) / (PPR * GEAR_RATIO * 4.0f *
SAMPLE_TIME));
            if (fabs(CurrentAngle) > 45.0f) {
                pwm=0;
                SumError = 0;
            } else {
                Error = TARGET_ANGLE - CurrentAngle;

```

```
SumError += Error * DT;
if(SumError > 200) SumError = 200;
if(SumError < -200) SumError = -200;
P= (Kp * Error);

                                I=(Ki * SumError);
                                D= (Kd * (-GyroRate));

PID_Output = P+I+D;
float DesiredSpeed = fabs(PID_Output);
if ( DesiredSpeed < 0.5 ) DesiredSpeed=0.5;
pwm = PIDVel(DesiredSpeed, CurVel);
if (PID_Output >= 0) {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_0, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_1, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_2, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_RESET);
} else {
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_0, GPIO_PIN_RESET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_1, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_3, GPIO_PIN_SET);
    HAL_GPIO_WritePin(GPIOE, GPIO_PIN_2, GPIO_PIN_RESET);
}
}
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_1, pwm);
__HAL_TIM_SET_COMPARE(&htim4, TIM_CHANNEL_2, pwm);
}
}
}
}
}
/* --- ĐỊNH NGHĨA HÀM --- */
void MPU6050_Init(void) {
    uint8_t data = 0x00;
    HAL_I2C_Mem_Write(&hi2c1, MPU_ADDR, 0x6B, 1, &data, 1, 100);
}
```

3.3.4 Tinh chỉnh thông số và Kết quả thực nghiệm.

3.3.4.1 Quá trình tinh chỉnh

- **Thông số lý thuyết ban đầu:**
 - Từ phương pháp Step Response và quy tắc Ziegler–Nichols, nhóm tính được $K_p = 12.29$, $K_i = 48.43$.
- **Quan sát thực nghiệm:**
 - Khi nạp vào vi điều khiển, động cơ bám theo vận tốc đặt nhưng thời gian đáp ứng quá chậm (5–10 giây).
 - Nguyên nhân: hệ số khuếch đại chưa đủ lớn, sai số trong quá trình đo thời gian trễ (L) và hằng số thời gian (Tau).
- **Tinh chỉnh thủ công (Trial and Error):**
 - Nhóm tăng dần K_p để cải thiện tốc độ phản ứng.
 - Điều chỉnh K_i để loại bỏ sai số tĩnh và giữ vận tốc ổn định.
 - Kết quả cuối cùng: **$K_p = 18.5$, $K_i = 150$, $K_d = 0$** cho vòng tốc độ.

3.3.4.2 Kết quả thực nghiệm

1. Vòng tốc độ (Speed Loop):

- **Trạng thái:** Đã thiết lập cấu trúc vòng lặp nhưng chưa đạt độ ổn định cần thiết khi kết hợp với vòng góc.
- **Hiện tượng:** Khi chạy độc lập (nhấc robot lên khỏi mặt đất), động cơ phản ứng nhanh với lệnh điều khiển. Tuy nhiên, khi đặt xuống đất, mô-men xoắn chưa đủ lớn để thắng quán tính cơ khí nhanh chóng, dẫn đến thời gian đáp ứng thực tế bị trễ.
- **Vấn đề:** Có hiện tượng trễ (latency) giữa lệnh điều khiển tốc độ và phản ứng thực tế của bánh xe, khiến robot không kịp bù góc nghiêng.

2. Vòng góc (Angle Loop) - Cấu trúc Cascade:

Nhóm đã tiến hành tinh chỉnh theo phương pháp Cascade nhưng chưa tìm được bộ thông số tối ưu:

- **K_p_angle :** Khi tăng K_p để robot đứng thẳng, hệ thống bắt đầu xuất hiện dao động cộng hưởng mạnh. Nếu K_p thấp, robot không đủ lực để chống lại trọng lực và đổ ngay lập tức.
- **K_d_angle :** Đã thử nghiệm thêm K_d để dập tắt dao động, nhưng do nhiễu từ cảm biến IMU (MPU6050) nên K_d cao gây hiện tượng rung (vibration) cực lớn ở động cơ thay vì làm mượt chuyển động.

- **Ki_angle:** Chưa thể áp dụng hiệu quả vì robot chưa duy trì được trạng thái cân bằng tạm thời để triệt tiêu sai số tĩnh.
- **Thông số thử nghiệm hiện tại:** $K_p \approx 0.15-0.25$; $K_d \approx 0.005-0.02$.

3. Đánh giá tổng quan (Thực tế):

- **Trạng thái hiện tại: Chưa đứng vững.** Robot có xu hướng đổ về một phía sau 1-2 giây sau khi thả tay.
- **Sai số:** Góc nghiêng vượt quá ngưỡng kiểm soát ($\pm 10^\circ$) dẫn đến mất thăng bằng hoàn toàn.
- **Nguyên nhân dự kiến:**
 1. Tần số lấy mẫu (Sample Rate) của vòng lặp PID có thể quá chậm (cần đạt $>100\text{Hz}$).
 2. Trọng tâm (Center of Mass) của robot quá cao so với thiết kế lý tưởng.
 3. Độ rơ (Backlash) của hộp số động cơ làm mất độ chính xác ở vùng cân bằng nhỏ.
 4. Nhiễu từ cảm biến IMU chưa được lọc tốt qua bộ lọc Complementary hoặc Kalman.

4. Kế hoạch khắc phục:

- Kiểm tra lại bộ lọc tín hiệu IMU để lấy góc nghiêng chính xác và mịn hơn.
- Tăng tốc độ vòng lặp điều khiển (Loop time).
- Thực hiện cân bằng cơ khí (điều chỉnh vị trí Pin) để tìm điểm cân bằng tĩnh tốt nhất trước khi chạy code

Chương IV. KẾT LUẬN.

4.1 Tổng kết quá trình nghiên cứu

Đề tài đã hoàn thành việc thiết kế và chế tạo mô hình xe cân bằng mini dựa trên hệ thống nhúng STM32F407, kết hợp cùng cảm biến MPU6050, Encoder và Driver TB6612FNG. Về mặt thuật toán, nhóm đã xây dựng và cài đặt thành công cấu trúc **PID lồng nhau (Cascade PID)** bao gồm:

- **Vòng trong (Speed Loop):** Kiểm soát vận tốc dựa trên phản hồi từ Encoder để đảm bảo động cơ hoạt động đúng dải tốc độ mong muốn.
- **Vòng ngoài (Angle Loop):** Thu thập và xử lý dữ liệu góc nghiêng từ bộ lọc bù (hoặc Kalman) để tính toán lực bù cân bằng.

Quá trình thực nghiệm cho thấy sự phức tạp trong việc phối hợp giữa hai vòng điều khiển. Kết quả thực tế giúp nhóm nhận thức rõ sự khác biệt giữa mô hình lý thuyết và các yếu tố phi tuyến trong thực tế như độ rơ hộp số, nhiễu cảm biến và trễ hệ thống.

4.2 Kết quả đạt được và Hạn chế tồn đọng

Kết quả đạt được:

- Hoàn thiện phần cứng robot: Cơ cấu cơ khí chắc chắn, hệ thống mạch điện hoạt động ổn định, không có hiện tượng sụt áp hay nhiễu tín hiệu điều khiển.
- Xử lý tín hiệu cảm biến: Dữ liệu từ MPU6050 được lọc nhiễu tốt, phản ánh chính xác góc nghiêng theo thời gian thực.
- Động cơ phản ứng nhạy: Hệ thống điều khiển đã xuất được xung PWM tương ứng với sự thay đổi góc nghiêng, động cơ có xu hướng chạy về phía robot để bù góc.

Hạn chế và Vấn đề cần giải quyết (Nguyên nhân robot chưa đứng được):

- **Sự ổn định:** Robot đã có phản xạ cân bằng nhưng chưa duy trì được trạng thái đứng yên lâu dài. Hiện tượng dao động quá mức (over-oscillation) vẫn xảy ra khi tăng hệ số K_p .
- **Tinh chỉnh PID:** Bộ thông số PID hiện tại mới chỉ dừng lại ở mức phản ứng tức thời, chưa đạt đến điểm cân bằng tối ưu giữa lực quán tính và lực bù. Sai số góc hiện tại vẫn dao động vượt ngưỡng kiểm soát ($\pm 10^\circ$) dẫn đến mất thăng bằng sau thời gian ngắn.
- **Yếu tố cơ khí:** Độ rơ (backlash) của động cơ DC giảm tốc và trọng tâm robot đặt chưa thực sự chuẩn xác gây khó khăn cho việc tinh chỉnh các thông số nhỏ (K_i , K_d).

NGUỒN THAM KHẢO